

Kody Huffmana (dokończenie) i szeregowanie zadań

1 Optymalne kody prefiksowe – przypomnienie

Przypomnijmy, że kody prefiksowe odpowiadają wzajemnie jednoznacznie *regularnym drzewom binarnym* (każdy wierzchołek wewnętrzny ma dokładnie dwoje dzieci), w których znaki alfabetu odpowiadają liściom.

Niech C będzie alfabetem, a

$$f : C \rightarrow \mathbb{N}$$

funkcją częstości (liczba wystąpień znaku). Dla drzewa T reprezentującego kod prefiksowy oznaczmy przez $d_T(c)$ głębokość liścia reprezentującego znak c w drzewie T . Głębokość liścia jest jednocześnie długością słowa kodowego znaku c .

Koszt drzewa T nazywamy liczbę bitów potrzebnych do zakodowania pliku:

$$B(T) = \sum_{c \in C} f(c) \cdot d_T(c) \longleftarrow \min.$$

Naszym celem jest znalezienie drzewa T minimalizującego $B(T)$ – takie drzewo nazywamy **optymalnym kodem prefiksowym**.

2 Własności optymalnego kodu prefiksowego

Lemat 1. Niech x i y będą najrzadziej występującymi znakami z C . Wtedy istnieje optymalny kod prefiksowy, w którym słowa kodowe x i y mają tę samą długość i różnią się tylko na ostatnim bicie.

Innymi słowy: istnieje optymalne drzewo, w którym liście reprezentujące x i y mają wspólnego rodzica i znajdują się na największej głębokości w drzewie.

Dowód. Znajdziemy drzewo reprezentujące optymalny kod prefiksowy, w którym liście reprezentujące x i y mają wspólnego rodzica i mają największą głębokość w drzewie.

Niech T będzie dowolnym drzewem reprezentującym optymalny kod prefiksowy i niech a oraz b będą dwoma liśćmi o tym samym rodzicu i największej głębokości w T (takie dwa liście istnieją, bo drzewo jest regularne, więc najgłębszy liść ma rodzeństwo).

Bez straty ogólności możemy przyjąć, że

$$f(x) \leq f(y) \quad \text{oraz} \quad f(a) \leq f(b).$$

Ponieważ $f(x)$ i $f(y)$ są najmniejszymi częstościami, to

$$f(x) \leq f(a) \quad \text{oraz} \quad f(y) \leq f(b).$$

Niech T' będzie drzewem powstałym z T przez zamianę miejscami liści x i a . Wówczas $d_{T'}(a) = d_T(x)$ oraz $d_{T'}(x) = d_T(a)$, a położenia pozostałych liści się nie zmieniają. Liczymy

różnicę kosztów:

$$\begin{aligned}
B(T) - B(T') &= \sum_{c \in C} f(c) d_T(c) - \sum_{c \in C} f(c) d_{T'}(c) \\
&= f(a) d_T(a) + f(x) d_T(x) - f(a) d_{T'}(a) - f(x) d_{T'}(x) \\
&= f(a) d_T(a) + f(x) d_T(x) - f(a) d_T(x) - f(x) d_T(a) \\
&= f(a)(d_T(a) - d_T(x)) - f(x)(d_T(a) - d_T(x)) \\
&= \underbrace{(f(a) - f(x))}_{\geq 0} \underbrace{(d_T(a) - d_T(x))}_{\geq 0} \geq 0.
\end{aligned}$$

Pierwszy czynnik jest nieujemny, bo $f(x) \leq f(a)$. Drugi czynnik jest nieujemny, bo a jest liściem o największej głębokości, więc $d_T(a) \geq d_T(x)$.

Zatem $B(T') \leq B(T)$, ale $B(T)$ jest najmniejszą możliwą wartością (bo T jest optymalne), więc $B(T') = B(T)$, czyli T' też reprezentuje optymalny kod prefiksowy.

Analogicznie zamieniamy miejscami liście y i b i otrzymujemy drzewo T'' reprezentujące optymalny kod prefiksowy, które ma żądane własności: liście x i y są rodzeństwem na największej głębokości. $\square$

Z Lematu 1 wynika, że konstrukcję optymalnego drzewa można rozpocząć od połączenia dwóch symboli o najmniejszej częstości – ich liście na pewno będą rodzeństwem w pewnym drzewie optymalnym.

Zachłanny wybór. W każdym kroku algorytm łączy dwa wierzchołki o najmniejszych częstościach w jeden nowy wierzchołek (poddrzewo), którego częstość jest sumą częstości połączonych wierzchołków.

Lemat 2. Niech A będzie alfabetem, a $f : A \rightarrow \mathbb{N}$ funkcją częstości. Niech x i y będą dwoma najrzadziej występującymi znakami w A . Niech

$$A' = (A \setminus \{x, y\}) \cup \{z\}, \quad z \notin A,$$

a $f' : A' \rightarrow \mathbb{N}$ będzie funkcją częstości taką, że dla każdego $c \in A' \setminus \{z\}$

$$f'(c) = f(c) \quad \text{oraz} \quad f'(z) = f(x) + f(y).$$

Niech T' będzie drzewem reprezentującym optymalny kod prefiksowy dla (A', f') . Niech T będzie drzewem powstałym z T' poprzez zastąpienie liścia z wierzchołkiem wewnętrznym z dziećmi x i y . Wówczas T reprezentuje optymalny kod prefiksowy dla (A, f) .

Dowód. Dla każdego $c \in A \setminus \{x, y\}$ mamy $d_T(c) = d_{T'}(c)$, natomiast

$$d_T(x) = d_T(y) = d_{T'}(z) + 1,$$

bo liście x i y pojawiają się o jeden poziom niżej niż dawny liść z .

Obliczamy koszt drzewa T :

$$\begin{aligned}
B(T) &= \sum_{c \in A} f(c) d_T(c) = \sum_{c \in A \setminus \{x, y\}} f(c) d_T(c) + f(x) d_T(x) + f(y) d_T(y) \\
&= \sum_{c \in A \setminus \{x, y\}} f'(c) d_{T'}(c) + f(x)(d_{T'}(z) + 1) + f(y)(d_{T'}(z) + 1) \\
&= \sum_{c \in A' \setminus \{z\}} f'(c) d_{T'}(c) + d_{T'}(z) (f(x) + f(y)) + (f(x) + f(y)) \\
&= \sum_{c \in A' \setminus \{z\}} f'(c) d_{T'}(c) + d_{T'}(z) f'(z) + (f(x) + f(y)) \\
&= \sum_{c \in A'} f'(c) d_{T'}(c) + f(x) + f(y) \\
&= B(T') + f(x) + f(y).
\end{aligned}$$

Stąd

$$B(T') = B(T) - f(x) - f(y).$$

Pozostaje pokazać, że T jest optymalne dla (A, f) . Przypuśćmy przeciwnie, że istnieje drzewo T_{opt} dla (A, f) takie, że $B(T_{\text{opt}}) < B(T)$. Na mocy Lematu 1 możemy założyć, że x i y są w T_{opt} rodzeństwem. Zastępując ich wspólnego rodzica liściem z otrzymujemy drzewo T'_{opt} dla (A', f') o koszcie

$$B(T'_{\text{opt}}) = B(T_{\text{opt}}) - f(x) - f(y) < B(T) - f(x) - f(y) = B(T'),$$

co przeczy optymalności T' dla (A', f') . Zatem T jest optymalne dla (A, f) . $\square$

3 Algorytm Huffmana

Lematy 1 i 2 uzasadniają poprawność następującego algorytmu zachłannego, który buduje optymalne drzewo „od dołu”, wielokrotnie łącząc dwa wierzchołki o najmniejszej częstości.

Algorithm 1 HUFFMAN(C, f) – budowa optymalnego kodu prefiksowego

Wejście: Alfabet C oraz funkcja częstości $f : C \rightarrow \mathbb{N}$

Wyjście: Korzeń drzewa optymalnego kodu prefiksowego

- 1: $Q \leftarrow$ kolejka priorytetowa zawierająca liście $c \in C$ z kluczem $f(c)$
 - 2: **for** $i = 1$ **to** $|C| - 1$ **do**
 - 3: $x \leftarrow \text{EXTRACT-MIN}(Q)$
 - 4: $y \leftarrow \text{EXTRACT-MIN}(Q)$
 - 5: utwórz nowy wierzchołek z z dziećmi x i y
 - 6: $f(z) \leftarrow f(x) + f(y)$
 - 7: $\text{INSERT}(Q, z)$
 - 8: **end for**
 - 9: **return** $\text{EXTRACT-MIN}(Q)$ {jedyne pozostałe wierzchołki – korzeń}
-

Przy użyciu kopca jako kolejki priorytetowej każda operacja EXTRACT-MIN i INSERT działa w czasie $O(\log |C|)$, więc cały algorytm działa w czasie $O(|C| \log |C|)$.

4 Szeregowanie zadań

Rozważmy teraz **prosty problem szeregowania zadań**. Dany jest zbiór zadań

$$\{z_1, z_2, \dots, z_n\},$$

gdzie każde zadanie z_i ma swój czas wykonania t_i , $i \in [n]$. Rozpoczęte zadanie nie może być przerwane (model bez wywłaszczania).

4.1 Problem 1 – jeden procesor

Problem 1. Mamy jeden procesor. Jak uszeregować zadania tak, aby *średni czas zakończenia* był jak najmniejszy?

Jeżeli zadania wykonywane są w kolejności $j_1, j_2, \dots, j_n$, to czas zakończenia k -tego zadania wynosi $C_k = \sum_{l=1}^k t_{j_l}$, a minimalizujemy

$$f(\Pi) = \frac{1}{n} \sum_{k=1}^n C_k = \frac{1}{n} \sum_{k=1}^n \sum_{l=1}^k t_{j_l}.$$

Przykład 1. Rozważmy cztery zadania:

i	zadanie	t_i
1	z_1	15
2	z_2	8
3	z_3	3
4	z_4	10

Dla kolejności z_1, z_2, z_3, z_4 średni czas zakończenia wynosi

$$\frac{15 + (15 + 8) + (15 + 8 + 3) + (15 + 8 + 3 + 10)}{4} = \frac{15 + 23 + 26 + 36}{4} = \frac{100}{4} = 25.$$

Dla kolejności z_3, z_2, z_4, z_1 (od najkrótszego do najdłuższego) otrzymujemy

$$\frac{3 + (3 + 8) + (3 + 8 + 10) + (3 + 8 + 10 + 15)}{4} = \frac{3 + 11 + 21 + 36}{4} = \frac{71}{4} = 17\frac{3}{4}.$$

Druga kolejność daje istotnie mniejszą wartość.

Rozwiązanie. Uszereguj zadania od najkrótszego do najdłuższego (rosnąco względem t_i).

Twierdzenie 1. Uszeregowanie zadań w kolejności niemalejących czasów wykonania minimalizuje średni czas zakończenia.

Dowód. Skorzystamy z *argumentu wymiany*. Niech $\Pi = (j_1, \dots, j_n)$ będzie permutacją minimalizującą f i przypuśćmy, że nie jest ona posortowana rosnąco. Wtedy istnieją dwa sąsiednie zadania j_k, j_{k+1} takie, że $t_{j_k} > t_{j_{k+1}}$.

Niech Π' powstaje z Π przez zamianę miejscami j_k i j_{k+1} . Wszystkie czasy zakończenia poza C_k pozostają bez zmian, a C_k zmienia się z wartości $(\sum_{l < k} t_{j_l}) + t_{j_k}$ na $(\sum_{l < k} t_{j_l}) + t_{j_{k+1}}$. Zatem

$$f(\Pi') - f(\Pi) = \frac{t_{j_{k+1}} - t_{j_k}}{n} \leq 0,$$

czyli $f(\Pi') \leq f(\Pi)$. Ale Π minimalizuje f , więc $f(\Pi') = f(\Pi)$ i Π' też minimalizuje f .

Po pewnej liczbie takich zamian otrzymujemy permutację $\Pi_0 = (j_1, \dots, j_n)$ minimalizującą f taką, że

$$t_{j_1} \leq t_{j_2} \leq \dots \leq t_{j_n}.$$

Permutacja posortowana rosnąco jest więc optymalna. □

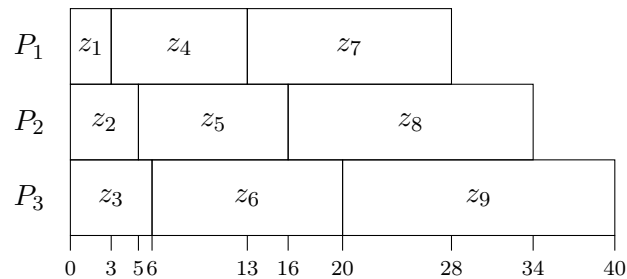
4.2 Problem 2 – wiele procesorów

Problem 2. Mamy 3 procesory. Pytanie jak poprzednio – minimalizujemy średni czas zakończenia.

Przykład 2. Rozważmy dziewięć zadań:

zadanie	z_1	z_2	z_3	z_4	z_5	z_6	z_7	z_8	z_9
t_i	3	5	6	10	11	14	15	18	20

Przydzielając zadania od najkrótszego do najdłuższego, każde do pierwszego wolnego procesora, otrzymujemy następujące uszeregowanie:



Procesory kończą pracę odpowiednio w chwilach 28 (P_1), 34 (P_2) i 40 (P_3).

Rozwiązanie. Przydzielaj zadania do pierwszego wolnego procesora w kolejności od najkrótszego do najdłuższego.

Wniosek 1. Nasze algorytmy szeregowania zadań (na jednym oraz na wielu procesorach) są poprawne – minimalizują średni czas zakończenia.

Poprawność wynika z tego samego argumentu wymiany: gdyby w optymalnym uszeregowaniu dwa zadania przydzielone do tego samego procesora były ułożone niezgodnie z porządkiem rosnącym czasów, ich zamiana nie zwiększyłaby średniego czasu zakończenia. Dlatego strategia zachłanna – najkrótsze zadanie do najwcześniej zwalnającego się procesora – daje rozwiązanie optymalne.